



CONTROL DE FLUJO: ESTRUCTURAS DE DECISIÓN

10145 - FUNDAMENTOS DE PROGRAMACIÓN PARA INGENIERÍA

10110 – FUNDAMENTOS DE COMPUTACIÓN Y PROGRAMACIÓN



RESUMEN DE CONTENIDOS

OPERADORES DE COMPARACIÓN

- Además de los operadores aritméticos, en Python existen los siguientes **operadores de comparación**

Mayor	>	4.3 > 3.2
Mayor o igual	>=	4.0 >= 4.1
Menor	<	-2 < 0
Menor o igual	<=	-3.14 <= -3.2
Igual	==	2.0 == 2
Distinto	!=	-2 != -2.1

OPERADORES LÓGICOS

- Existen además **operadores lógicos**:

- La **negación** (**not**) para **invertir** el valor de verdad

not 50 > 4 $\square$ **not** True $\square$ False

- La **conjunción** o “y lógico” (**and**), que resulta verdadero si y solo si **todas** las sub-expresiones son verdaderas

x > **y** **and** **y** <= **z**

- La **disyunción** u “o lógico” (**or**), que resulta verdadero si **al menos una** de las sub-expresiones lo es

x != **y** **or** **x** <= **z** **or** **y** < **z**



PRECEDENCIA DE LOS OPERADORES

- **Precedencia:** el orden en que Python evalúa una expresión
 1. Operaciones aritméticas
 2. Operadores de comparación
 3. Negación (not)
 4. Conjunción (and)
 5. Disyunción (or)
- **Se puede alterar con paréntesis**, igual que en aritmética
 - `5 + 3 > 6 and not 2 > 4`
 - `8 > 6 and not False`
 - `True and True`
 - `True`

COMPARACIONES ENCADENADAS

- **En este curso no se encadenan comparaciones**
 - Python permite escribir `a == b == c`, pero no significa `(a == b) == c`
 - Significa `a == b and b == c`: hay un `and` invisible que no está escrito
- **Con valores concretos:** si `a = 2`, `b = 2` y `c = 1`
 - `a == b == c` entrega `False`
 - `(a == b) == c` entrega `True`
- **La forma del curso:** una comparación por operador, unidas con operadores lógicos
 - `a == b and b == c`
 - `18 <= edad and edad <= 35`

SENTENCIA **if**

- Permite **condicionar la ejecución** de un bloque de sentencias a que una expresión booleana retorne **True**
 - Sintaxis:

```
if <booleano>:  
    # Se ejecuta si la condición se cumple  
    <Bloque de sentencias condicionales>  
<Bloque de sentencias que sigue>
```

- El bloque condicionado debe estar **indentado**. Para volver al flujo no condicionado es necesario volver al nivel previo de **indentación**

SENTENCIA **if-else**

- Permite **dividir el flujo de la ejecución de dos bloques** a que una expresión booleana retorne **True**
 - Sintaxis:

```
if <booleano>:  
    # Se ejecuta si la condición se cumple  
    <Bloque de sentencias condicionales>  
else:  
    # Se ejecuta si la condición no se cumple  
    <Bloque de sentencias alternativo>  
# Se ejecuta independiente de si la condición se cumplió o no  
<Bloque de sentencias que sigue>
```



SENTENCIA `if-elif-else`

– Sintaxis:

`if` <booleano>:

Se ejecuta si la condición 1 se cumple

<Bloque de sentencias condicionales 1>

`elif` <booleano>:

Se ejecuta si la condición 1 no se cumple y sí se cumple la condición 2 (Se puede crear tantos bloques `elif` como sea necesario, cada uno con una condición distinta)

<Bloque de sentencias condicionales 2>

`else`:

Se ejecuta si las condiciones 1 y 2 no se cumplen

<Bloque de sentencias alternativo>

Se ejecuta independiente de si la condición se cumplió o no

<Bloque de sentencias que sigue>



EJERCICIOS

EJERCICIO PROPUESTO 1

- Construye un programa que solicite el valor de dos ángulos interiores de un triángulo e indique si un triángulo es equilátero, isósceles o escaleno
 - Recuerda que los ángulos interiores siempre sumarán 180° .
 - Las entradas siempre serán números enteros positivos con valores de ángulos válidos.

EJERCICIO PROPUESTO 2

- Construye un programa que reciba una cantidad de dinero y entregue el desglose de este en billetes y monedas válidos en circulación en Chile (20.000, 10.000, 5.000, 2.000, 1.000, 500, 100, 50, 10, 5, 1)
- Por ejemplo, si la entrada fuese \$20.250, el programa debería entregar:
 - 1 billete(s) de \$20.000
 - 2 moneda(s) de \$100
 - 1 moneda(s) de \$50



¿CONSULTAS?